

Pedestrian crossing detection is one of many subproblems that need to be solved in order to allow for autonomous vehicles to drive safely and avoid injuring pedestrians. This report will discuss how to detect pedestrian crossing using a combination of rudimentary computer vision techniques and the reason for employing each technique.

Theory:

Before I start, I will provide the original image before doing any transformations on it, in order to show what each step of the processing does.

I first started by applying a **median filter** to remove some of the noise that is present in the images. After a bit of testing, a size 5x5 filter was shown to produce the best noise removal without blurring the pedestrian crossing too much. As shown in figure 2, the pedestrian crossings are now more vibrant while the ground noise, due to the rough surface and leaves, has been removed.

A normal binary threshold will work well in daylight, but will fail during night time. So, to combat this, **CLAHE**(Contrast Limited Adaptive Histogram Equalization), is used to enhance the contrast of the images, which increases the luminosity of pedestrian crossing in dark conditions. This is done by changing the images from BGR format into HLS format. Then, the channels are split from the images. CLAHE is specifically applied to the luminance channel and then the modified luminance channel is merged with the other channels in order to produce the equalised images.

Figure 1: Original image



Figure 2: Median Filter



Figure 3: Equalized Image



The images are transformed into **grayscale** next for further processing.

Even though CLAHE is quite useful, it introduces a bit of noise into the images. So, to combat this effect, another **median filter** is applied in order to remove a bit of the introduced noise. The filter is also a 5x5 filter.

In order to isolate the pedestrian crossings, thresholding should be applied. But, due to the various lighting conditions present, a simple threshold will not suffice and will produce uncertain results. In order to overcome this obstacle, **adaptive thresholding** was used. After some tinkering, It was found that a Gaussian kernel of size 89x89 with a C=1 produced the best result by isolating the pedestrian crossings from the rest of the image. However, this introduced some noise into the image, as shown in figure 6, which will be dealt with later in the processing phase.

Intuitively, one operation that is great at removing noise is the morphological operation of **Opening**. After a bit of trial and error, the 9x9 kernel proved to be the best at removing noise without eroding the pedestrian crossings. Figure 7 shows this.

After thresholding, **Connected component analysis** was performed in order to identify the regions and store them in an array for further processing. Moreover, any region with an area below 400 pixels was removed, since they were highly likely to be noise. Each region was assigned a random colour. This is shown in figure 8.

Figure 4: GrayScaled Image



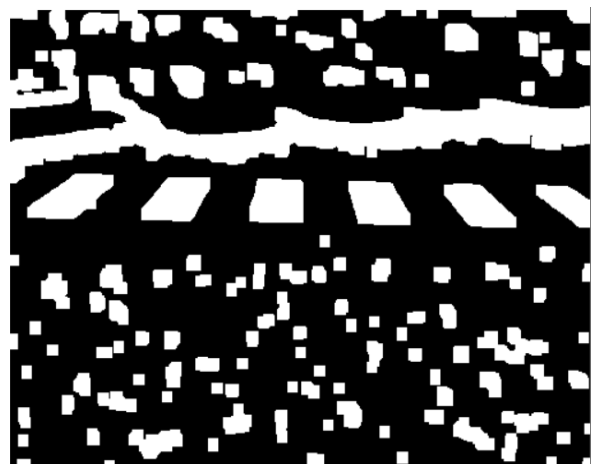
Figure 5: GrayScaled Median Image



Figure 6: Adaptive Thresholded Image



Figure 7: Opened Image



Before the processing continues, one of the assumptions present is that pedestrian crossings are always rectangular. Armed with this assumption, it is safe to use the **rectangularity** of a region in order to determine whether it is a pedestrian crossing or not. A region is bounded by a green rectangle iff its rectangularity is above 0.6, else it will be bounded by a red rectangle. The value of 0.6 was found to be the most accurate in distinguishing between pedestrian crossings and the noise present. If the value was above 0.6, a considerable amount of pedestrian crossings were marked as noise, since the contours of them messed with how the bounding rectangle was calculated. Even though this method gets rid of a bit of noise, some noise still persists due to the fact that they coincidentally have a high value for the rectangularity. Figure 10 highlights the clarity of the pedestrian crossings and the disappearance of most of the noise.

After that, **Robert's edge detection** was applied in order to extract the edges from the pedestrian crossings. This was done by defining Robert's Cross Operator Kernels, applying them both on the images and then merging both the results into a new Matrix. Figure 11 shows the result of this operation. The edges will be used in order to detect where exactly the pedestrian crossings are.

Another assumption that is present is that pedestrian crossings are somewhat horizontal to the car/vehicle that is present. This assumption will be used when the Hough Transform is used in order to highlight the pedestrian crossings, but before that, the images need to be in binary before being processed by the hough transform. Figures

Figure 8: CCA Image



Figure 9: Bounded Rectangularity Image

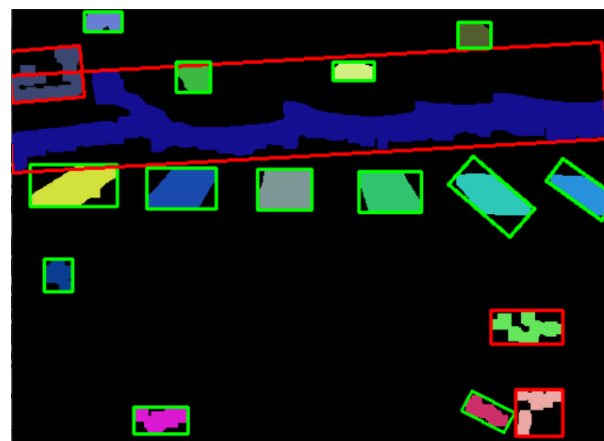
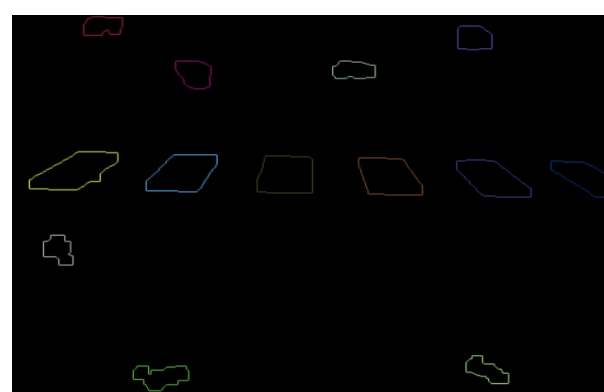


Figure 10: Red Rectangles Removed Image



Figure 11: Roberts Edge Image



11,12 and 13 show an image being changed from **BGR, to Grayscale and finally to Binary**.

The **Hough Transform** is applied on the binary edge images in order to outline the pedestrian crossings. This is possible due to the fact that pedestrian crossings are always aligned in some way, thus they have a lot of edge points in a straight line. After some testing, it was found that the minimum number of edge points that needed to be aligned to draw the line that proved to be the best was 55. This threshold was able to isolate the pedestrian crossings in most of the cases without drawing lines at noise edges. However, this was not enough to isolate the required lines. Since we assumed that pedestrian crossings tend to be horizontal-like. We can deduce that for certain thetas that indicate a vertical line, it is almost certainly not a pedestrian crossing. This observation led to the setting of a range, which after a bit of tweaking, became the set (1.2,1.8). This arises due to the fact that a horizontal line has a $\theta = \pi/2 \approx 1.5$. Another issue that we have to deal with is that sometimes, the hough transform draws lines that are extremely close to each other. Although these redundant lines highlight the strong lined up edges, they are unnecessary. One of the easiest ways to remove them is to sort the lines based on the theta values first, and if they are the same, sort them based on the rho values. After that, for every line that is drawn, search all the lines before it, and check if the lines are similar. Similar in this context is if the absolute difference between the 2 line's rho values is strictly smaller than 6 and their theta's absolute difference is strictly smaller than 0.2. These thresholds provided the best result when dealing with removing redundant lines without removing important ones. Figure 14 shows the resulting lines.

Figure 12: Roberts Grayscaled Edge Image

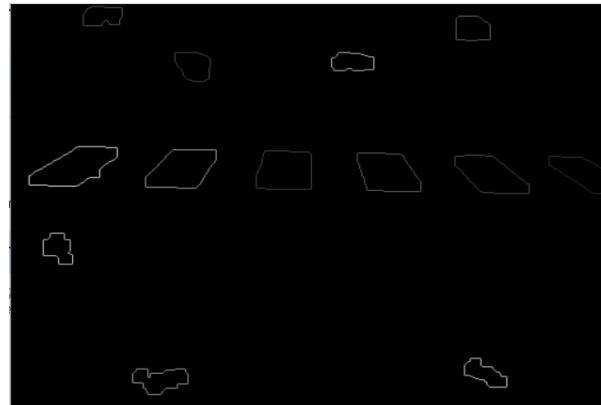


Figure 13: Roberts Binary Edge Image

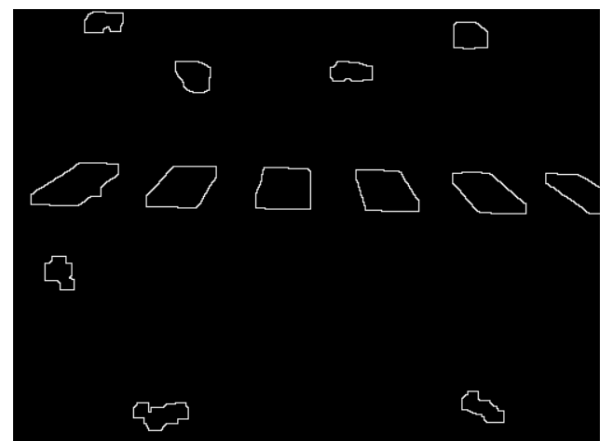


Figure 14: Hough Lines Image

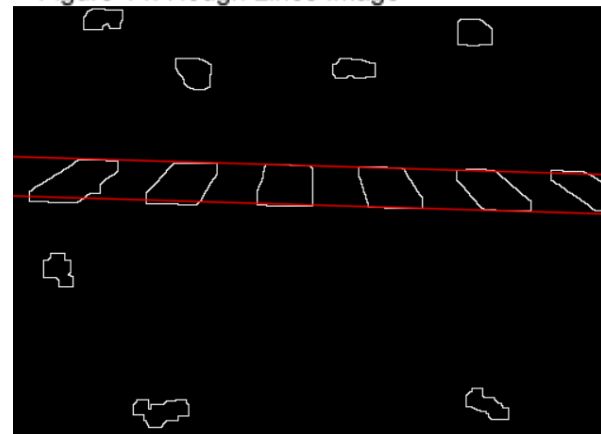


Figure 15: Hough Original Image



The lines that are left after the processing will be drawn on the original images. This, in theory, will outline the pedestrian crossings in the images. In addition to that, the ground truth, which is derived manually, is also drawn on the images. This will be useful in determining the effectiveness of our algorithm.

Figure 15: Hough Original Image



Evaluation:

One metric that I found to make a bit of sense in quantizing the performance of my algorithm is using the IoU metric. The way I did it was by counting pixels. If a pixel, at a certain location, had both max values for green and red pixels (The red lines are in a matrix and the green lines are in another one), then both the number of intersected pixels and union pixels are incremented. Else if either one of the max values is detected, the union pixels are incremented. This takes account of the intersection and the number of detected lines. The higher the IoU value, the lower the number of detected lines and the more they intersect with the ground truth. Even though the ground truth is subject to question, it stands as a good basis to measure the accuracy of the algorithm. [Here](#) are the detected and the ground truth images. The table below shows the IoU for the 20 images:

Image Number:	Intersection Pixels:	Union Pixels:	IoU:
1	1623	9531	0.170286
2(illustration Image)	2080	3968	0.524194
3	974	5074	0.191959
4	2555	3493	0.731463
5	724	5324	0.135988
6	559	6785	0.0823876
7	0	3024	0
8	1124	6249	0.179869
9	1344	4704	0.285714
10	0	847	0
11	1976	4072	0.485265

12	1826	8452	0.216044
13	1722	4326	0.398058
14	2019	5256	0.384132
15	2092	5468	0.38259
16	0	4536	0
17	1150	6280	0.183121
18	1653	7299	0.226469
19	0	7871	0
20	0	0	NaN

After analysing the IoU values and comparing them with the images, I found that the images with the IoU values > 0.1 gave acceptable results and showed great similarity between the ground truth and the detected lines, taking into account the false positives that were sometimes detected. IoU values > 0.2 show a remarkable similarity between the detected and the truth and even pointed out the non-existence of false positives. IoU values of 0 indicated that either the detected lines were far away from the ground truth, or that detected lines were present when no ground truth was needed, since no pedestrian crossings were there. So, Iou of 0 indicated false positives. NaN iou shows that no pedestrian crossings were present and that no detected lines were present, thus it is a desirable value to receive.

Other measures that could be used to show the performance of the algorithm are the Recall, Precision and the F_1 . The TP,FP and FN will be manually measured and assigned according to my judgement. This might be a source of error, but it provides a fast way of computing the data. An assumption that I made was that if there are redundant lines, then they will be counted as FPs . The table below shows the TP,FP,FN,Recall,Precision and F_1 for the 20 images:

Image Number:	TP:	FP:	FN:	Recall:	Precision:	F_1 :
1	2	5	0	1.000	0.286	0.445
2	2	0	0	1.000	1.000	1.000
3	2	0	0	1.000	1.000	1.000
4	2	0	0	1.000	1.000	1.000
5	2	0	0	1.000	1.000	1.000

6	1	2	1	0.500	0.333	0.400
7	0	0	2	0.000	NaN	NaN
8	2	1	0	1.000	0.667	0.800
9	2	0	0	1.000	1.000	1.000
10	0	1	0	NaN	0.000	NaN
11	2	0	0	1.000	1.000	1.000
12	2	3	0	1.000	0.400	0.571
13	2	0	0	1.000	1.000	1.000
14	2	1	0	1.000	0.667	0.800
15	2	1	0	1.000	0.667	0.800
16	0	1	2	0.000	0.000	NaN
17	1	2	1	0.500	0.333	0.400
18	3	1	1	0.750	0.750	0.750
19	0	6	0	NaN	0.000	NaN
20	0	0	0	NaN	NaN	NaN
Averages				0.809	0.617	0.798

Excluding NaNs, the average F_1 measure is 0.798 which speaks highly of the performance of the algorithm, it indicates a balance between the recall and the precision, which means that their values are not too dissimilar on average. An average recall of 0.809 is pretty good. It shows that the algorithm detects the pedestrian crossings very consistently while having an acceptable precision metric, which means that the algorithm does not hallucinate and produce a lot of false positives, but there seems to be a considerable amount of false positives scattered in some images.

NaNs indicate both failures and successes. When both the Recall and the Precision are NaN, it shows that the algorithm was successful in showing that no pedestrian crossings exist in the image. However, if only one of them has the NaN value, then it indicates something wrong has occurred. When the Recall only has the NaN value, it means that there were no pedestrian crossings in the image, but the algorithm drew false positives in the image, and when the precision is NaN, it means that there were pedestrian crossings in the image, but the algorithm failed to detect them. If both the Recall and the Precision are 0, then the algorithm hallucinated a false positive and did not detect the pedestrian crossing in the image.

One of the noticeable failures in the algorithm is that it seems to not be able to detect diagonal pedestrian crossings, as seen in the image set. There are two explanations for this failure. Firstly, the theta range which limits the drawing of non-horizontal-like lines might be interfering with the drawing of the outline lines for the pedestrian crossings. A fix is to tinker with this range a bit in order to find a more inclusive range. Another reason is that their diagonal position might make them more susceptible to the Erosion operation in the Opening. Playing with the size of the Opening kernel might fix that, but maybe at the cost of an increase in the amount of noise in the preprocessing stage.

Another failure in the algorithm is the presence of false positives in the image. This is caused by the presence of noise in the Robert's Binary Edge Detection section of the preprocessing stage, which coincidentally line up in a linear way, resulting in a hough line being drawn through these noise edges. This noise could be attributed to the use of the adaptive threshold technique. A fix is to use another thresholding technique, like setting a static threshold, but this will come at the expense of the robustness of the technique, since it is able to work at night time due to the adaptive threshold. Another possible fix is to maybe use to count the number of convex hulls in any given contour and set a threshold for the convex hulls. Noise has a higher probability of having an irregular shape than pedestrian crossings, so would, in theory, have a higher number of convex hulls. Tweaking with the other parameters in the algorithm might also lead to a higher precision rate.

A different technique that could be utilised in the algorithm is the usage of canny edge detection in or to identify the prominent edges in the images. Of Course the various thresholds of it would have to be tinkered with in order to produce acceptable results. In addition to that, Multispectral edge detection could be used in order to highlight the desired edges in the image. Some post processing might be in order, but nonetheless it is a viable route to go through.

In the end, the pedestrian crossing detection problem is a pretty tricky problem to deal with. More research and development is definitely needed in order to increase the Recall and Precision of said algorithm. Hopefully, this report serves as a basis for the solution of this real life problem.